

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Linux Firewall and Secure Remote Access

Laboratory 4: Practical Laboratory Report

CORE102 TCP/IP Networking and Linux Network Tools

Student name	Muhammad Haruna Osori
Registration number	C11/26/OCO/17007
Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Laboratory	Laboratory 4, Linux Firewall and Secure Remote Access (maximum 100 marks)
Submission date	

Evidence index

The manual lists six items of required evidence. This table shows where each one can be found in the report.

Item	Required evidence	Report section	Included
1	Baseline service and socket inventory	Part 1	Yes
2	UFW policy and numbered-rule evidence before and after testing	Parts 3 and 8	Yes
3	Client-side allowed and denied connection tests	Parts 2 and 4	Yes
4	Safe iptables custom-chain exercise and cleanup	Part 5	Yes
5	SSH public-key setup, fingerprint, successful login and server validation	Parts 6 and 7	Yes
6	Final exposure review and rollback record	Part 8	Yes

1. Purpose and Laboratory Environment

I used two virtual machines on a host-only network. The addresses in the manual (192.168.56.10 and 192.168.56.20, and the account name student) are examples, so wherever they appear in the manual I substituted the values from my own setup, listed below.

System	Hostname and account	IP address	Role
Server VM	ubuntu-server, user ubuntu	192.168.56.102	Ubuntu Linux. Runs UFW, the service inventory and SSH hardening
Client VM	kali, user kali	192.168.56.101	Kali Linux. Used for connectivity tests and SSH key authentication
Private network	Host-only	192.168.56.0/24	Isolated lab network, separate from any other network

All work was done inside this authorised laboratory environment. No private key or password appears anywhere in this report. Only the public key fingerprint is shown.

2. Part 1: Baseline and Recovery Preparation

2.1 Recovery preparation

Before touching any security setting I took a snapshot of the server VM and confirmed that I could open its console from the hypervisor. The console matters because a wrong firewall or SSH change can cut off remote access, and the console is the only way back in that does not depend on the network. I also checked that the Kali client could reach the server before any rules were applied.

2.2 System identification

I created the working folder `~/core102_lab4` and recorded the date, hostname, interfaces and routes. The commands ran on ubuntu-server on Monday 28 September 2026 at 22:29 UTC.

```

File  Machine  View  Input  Devices  Help
kali@kali: ~/co

Session  Actions  Edit  View  Help

ubuntu@ubuntu-server:~/core102_lab4$ date
Mon Sep 28 10:29:10 PM UTC 2026
ubuntu@ubuntu-server:~/core102_lab4$ hostname
ubuntu-server
ubuntu@ubuntu-server:~/core102_lab4$ ip -br addr
lo                UNKNOWN    127.0.0.1/8 ::1/128
enp0s3            UP          fe80::a00:27ff:feb9:3d84/64
enp0s8            UP          192.168.56.102/24 metric 1024 fe80::a00:27ff:feb9:
fd97/64
ubuntu@ubuntu-server:~/core102_lab4$ ip route
default via 192.168.56.1 dev enp0s8 proto dhcp src 192.168.56.102 metric 1024
1.1.1.1 via 192.168.56.1 dev enp0s8 proto dhcp src 192.168.56.102 metric 1024
8.8.8.8 via 192.168.56.1 dev enp0s8 proto dhcp src 192.168.56.102 metric 1024
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.102 metric 1024
192.168.56.1 dev enp0s8 proto dhcp scope link src 192.168.56.102 metric 1024
ubuntu@ubuntu-server:~/core102_lab4$ sudo ss -tulpn
[sudo: authenticate] Password:
Netid  State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
Process
udp    UNCONN  0        0        127.0.0.54:53        0.0.0.0:*
users:((("systemd-resolve",pid=472,fd=18))
udp    UNCONN  0        0        127.0.0.53%lo:53     0.0.0.0:*
users:((("systemd-resolve",pid=472,fd=16))
udp    UNCONN  0        0        0.0.0.0:67           0.0.0.0:*
users:((("dhcpd",pid=1122,fd=7))
udp    UNCONN  0        0        192.168.56.102%enp0s8:68 0.0.0.0:*
users:((("systemd-network",pid=585,fd=33))
udp    UNCONN  0        0        127.0.0.1:323        0.0.0.0:*
users:((("chronyd",pid=958,fd=4))
udp    UNCONN  0        0        [::]:323             [::]:*
users:((("chronyd",pid=958,fd=5))
tcp    LISTEN  0        4096     127.0.0.54:53        0.0.0.0:*
users:((("systemd-resolve",pid=472,fd=19))
tcp    LISTEN  0        4096     127.0.0.53%lo:53     0.0.0.0:*

```

Figure 3: date, hostname, ip -br addr, ip route and the start of ss -tulpn on the server.

- **Interfaces.** The loopback interface has 127.0.0.1. The interface enp0s8 is up with 192.168.56.102/24, and this is the lab network. The interface enp0s3 is also up but shows only an IPv6 link-local address.
- **Routing.** The default route goes through 192.168.56.1 on enp0s8, and 192.168.56.0/24 is directly connected. DHCP also installed host routes for 1.1.1.1 and 8.8.8.8 through the same gateway.

2.3 Listening sockets

I used `sudo ss -tulpn` to list every listening TCP and UDP socket together with the process that owns it. The first screenshot above cuts off partway through the TCP entries, so I repeated the command to capture the whole output.

```

ubuntu@ubuntu-server:~/core102_lab4$ sudo ss -tulpn
[sudo: authenticate] Password:
Netid State  Rcvd-Q  Send-Q      Local Address:Port      Peer Address:Port
Process
udp    UNCONN  0        0      127.0.0.54:53            0.0.0.0:*
users:(( "systemd-resolve",pid=472,fd=18))
udp    UNCONN  0        0      127.0.0.53%lo:53        0.0.0.0:*
users:(( "systemd-resolve",pid=472,fd=16))
udp    UNCONN  0        0        0.0.0.0:67              0.0.0.0:*
users:(( "dhcpd",pid=1122,fd=7))
udp    UNCONN  0        0     192.168.56.102%enp0s8:68 0.0.0.0:*
users:(( "systemd-network",pid=585,fd=33))
udp    UNCONN  0        0      127.0.0.1:323           0.0.0.0:*
users:(( "chronyd",pid=958,fd=4))
udp    UNCONN  0        0      [::]:323                [::]:*
users:(( "chronyd",pid=958,fd=5))
tcp    LISTEN  0        4096      127.0.0.54:53            0.0.0.0:*
users:(( "systemd-resolve",pid=472,fd=19))
tcp    LISTEN  0        4096      127.0.0.53%lo:53        0.0.0.0:*
users:(( "systemd-resolve",pid=472,fd=17))
tcp    LISTEN  0        4096        0.0.0.0:22              0.0.0.0:*
users:(( "sshd",pid=1350,fd=3),("systemd",pid=1,fd=100))
tcp    LISTEN  0        4096        [::]:22                  [::]:*
users:(( "sshd",pid=1350,fd=4),("systemd",pid=1,fd=101))
ubuntu@ubuntu-server:~/core102_lab4$ sudo systemctl --type=service --state=running
--no-pager

```

Figure 4: Complete `sudo ss -tulpn` output on the server, including the SSH listeners on port 22.

From this output I built the service inventory below.

Port and protocol	Bind address	Process	Expected purpose	Keep or remove
22/tcp	0.0.0.0 and	sshd (pid 1350)	Remote administration over SSH	Keep. Restrict to the lab subnet with UFW and harden with key authentication
53/tcp, 53/udp	127.0.0.53 and 127.0.0.54	systemd-resolved (pid 472)	Local DNS stub resolver	Keep. Loopback only, not reachable from other hosts
67/udp	0.0.0.0	dhcpd (pid 1122)	DHCP server for the lab network	Keep only if this server is meant to issue leases. Listens on all interfaces, so it needs a justification
68/udp	192.168.56.102 (enp0s8)	systemd-network (pid 585)	DHCP client that obtained the server's own address	Keep
323/udp	127.0.0.1 and [::1]	chronyd (pid 958)	Local control port of the NTP time client	Keep. Loopback only

The bind address decides who can reach a service. SSH on port 22 and DHCP on port 67 are bound to all interfaces, so they are the services other machines on the network can actually reach. SSH is also listening on IPv6. Nothing was listening on port 8080 at this stage, which is expected because the test web service does not exist. The port 22 sockets also list `systemd` as an owner, which tells me SSH is started through `systemd` socket activation.

2.4 Running services

I then listed the running services with `systemctl --type=service --state=running`. The system reported 24 running services.

```

ubuntu@ubuntu-server:~/core102_lab4$ sudo systemctl --type=service --state=running
--no-pager
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
chrony.service                     loaded active running chrony, an NTP client/server
cron.service                       loaded active running Regular background program pr...
dbus.service                       loaded active running D-Bus System Message Bus
fwupd.service                      loaded active running Firmware update daemon
getty@tty1.service                 loaded active running Getty on tty1
isc-dhcp-server.service            loaded active running ISC DHCP IPv4 server
ModemManager.service              loaded active running Modem Manager
multipathd.service                loaded active running Device-Mapper Multipath Devic...
networkd-dispatcher.service        loaded active running Dispatcher daemon for systemd...
NetworkManager.service            loaded active running Network Manager
polkit.service                    loaded active running Authorization Manager
rsyslog.service                   loaded active running System Logging Service
snapd.service                     loaded active running Snap Daemon
ssh.service                       loaded active running OpenBSD Secure Shell server
systemd-journald.service           loaded active running Journal Service
systemd-logind.service             loaded active running User Login Management
systemd-networkd.service           loaded active running Network Management
systemd-resolved.service           loaded active running Network Name Resolution
systemd-udevd.service              loaded active running Rule-based Manager for Device...
udisks2.service                   loaded active running Disk Manager
unattended-upgrades.service         loaded active running Unattended Upgrades Shutdown
upower.service                    loaded active running Daemon for power management
user@1000.service                 loaded active running User Manager for UID 1000
wpa_supplicant.service             loaded active running WPA supplicant

Legend: LOAD    → Reflects whether the unit definition was properly loaded.
          ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
          SUB    → The low-level unit activation state, values depend on unit type.

24 loaded units listed.
ubuntu@ubuntu-server:~/core102_lab4$ sudo ufw status verbose

```

Figure 5: Running services on the server (24 units).

Two of these accept connections from the network: `ssh.service` and `isc-dhcp-server.service`. The rest handle networking, time, logging, scheduling and updates. A few, such as `ModemManager`, `wpa_supplicant` and `udisks2`, are not needed on a virtual server and would be candidates for removal on a production host. I did not change any services in this lab, because the purpose of Part 1 is to record a baseline.

2.5 Firewall status

```

ubuntu@ubuntu-server:~/core102_lab4$ sudo ufw status verbose
Status: inactive
ubuntu@ubuntu-server:~/core102_lab4$ sudo iptables -L -n -v --line-numbers
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
ubuntu@ubuntu-server:~/core102_lab4$ sudo ip6tables -L -n -v --line-numbers 2>/dev
/null || true
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
ubuntu@ubuntu-server:~/core102_lab4$

```

Figure 6: UFW status, iptables and ip6tables before any rules were configured.

UFW reported Status: inactive. The INPUT, FORWARD and OUTPUT chains in both iptables and ip6tables had a default policy of ACCEPT, no rules, and zero packets and bytes counted.

Conclusion. The server had no packet filtering at all. SSH and DHCP were reachable by any host that could route to the server. The remaining parts of the lab close that gap.

3. Part 2: Controlled Test Service

To have something to allow and block, I started a temporary web server with Python in a first terminal on the server. It serves a one-line page and listens on TCP port 8080 on all interfaces.

```

mkdir -p ~/core102_lab4/web
echo 'CORE102 authorised firewall test' > ~/core102_lab4/web/index.html
cd ~/core102_lab4/web
python3 -m http.server 8080 --bind 0.0.0.0

```

That terminal stays busy while the server runs, so I opened a second terminal on the server and confirmed the listener with `sudo ss -ltnp | grep ':8080'`.

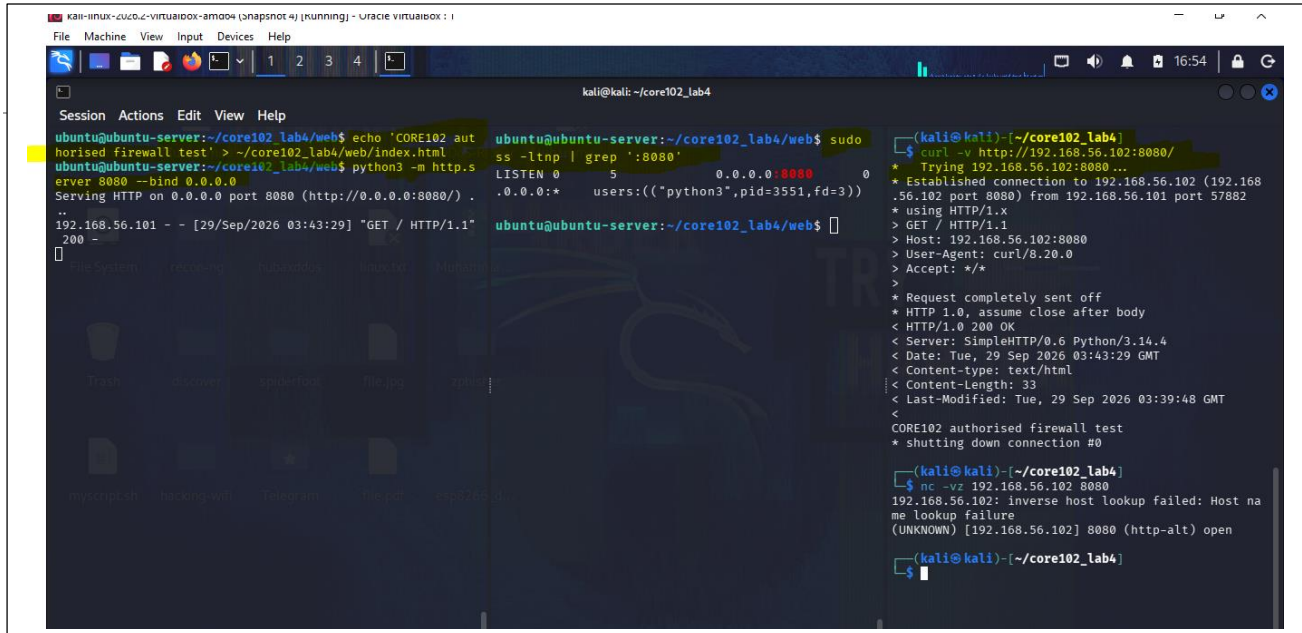


Figure 7: The Python web server listening on TCP port 8080 (server, second terminal).

From the Kali client I then tested the connection. Netcat reported the port as open:

```
(kali@kali)-[~/core102_lab4]
└─$ nc -vz 192.168.56.102 8080
192.168.56.102: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.56.102] 8080 (http-alt) open
```

The reverse lookup line is only a warning. Netcat tried to turn the IP address into a hostname and found none, which is normal on a private lab network with no DNS entry for the server. The last line is the result that matters: port 8080 is open and reachable from the client.

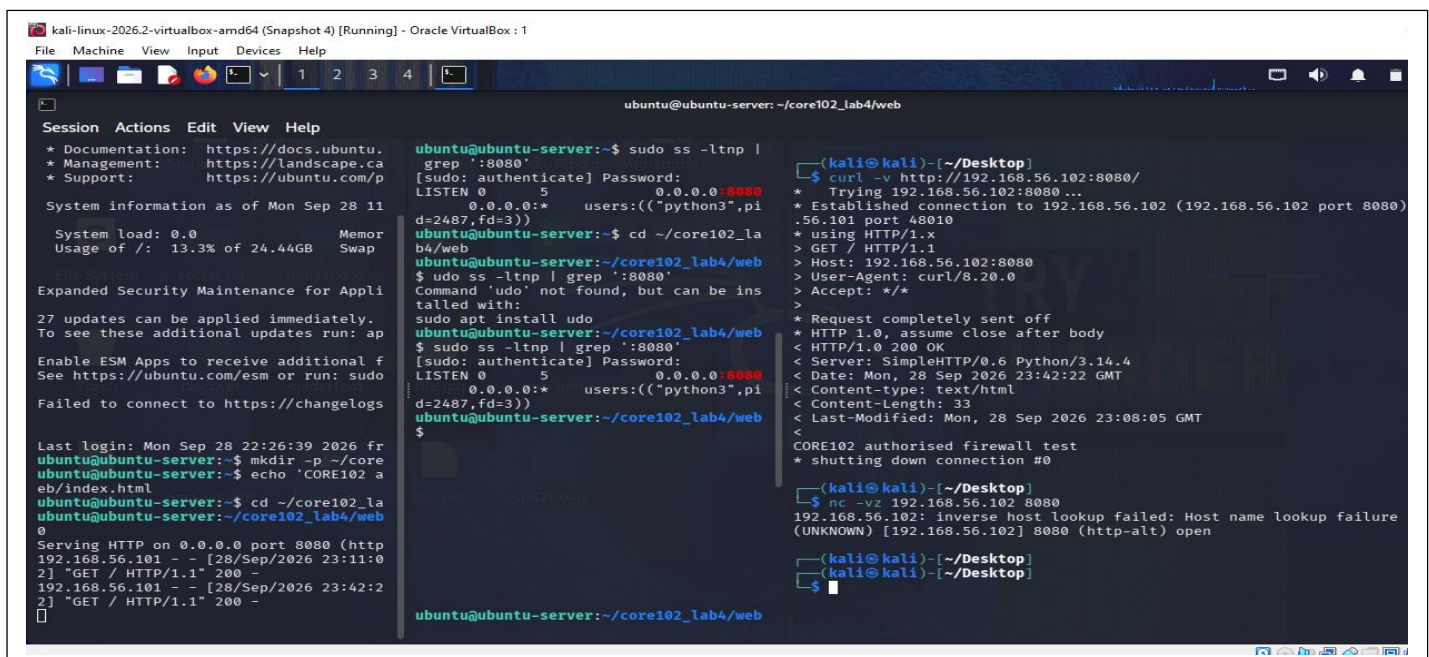


Figure 8: Client tests of the web service, nc and curl.

The curl request returns the text CORE102 authorised firewall test, which confirms the service answers end to end.

4. Part 3: UFW Policy Implementation

I built the policy in an order that could not lock me out. SSH was allowed before the firewall was switched on, and I kept my existing session open while I worked. The steps were:

```
sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment 'CORE102 SSH'
sudo ufw allow from 192.168.56.20 to any port 8080 proto tcp comment 'CORE102 test web'
sudo ufw allow from 192.168.56.101 to any port 8080 proto tcp comment 'CORE102 test web'
sudo ufw deny 23/tcp comment 'Block Telnet'
sudo ufw enable
sudo ufw status numbered
sudo ss -ltnp
```

```
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw status verbose
[sudo: authenticate] Password:
Status: inactive
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment 'CORE102 SSH'
Rules updated
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw allow from 192.168.56.20 to any port 8080 proto tcp comment 'CORE102 test web'
Rules updated
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw deny 23/tcp comment 'Block T
```

```
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw deny 23/tcp comment 'Block T
elnet'
Rules updated
Rules updated (v6)
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw enable
Command may disrupt existing ssh connec
tions. Proceed with operation (y/n)? y
Firewall is active and enabled on syste
m startup
```

Figure 9: UFW default policies, rule creation and activation.

The numbered rule list I captured while testing was:

Status: active

	To	Action	From	
	--	-----	----	
[1]	8080/tcp	ALLOW IN	Anywhere	
[2]	80/tcp	ALLOW IN	Anywhere	
[3]	22/tcp	ALLOW IN	192.168.56.0/24	# CORE102 SSH
[4]	8080/tcp	ALLOW IN	192.168.56.20	# CORE102 test web
[5]	23/tcp	DENY IN	Anywhere	# Block Telnet
[6]	8080/tcp	ALLOW IN	192.168.56.101	# CORE102 test web
[7]	8080/tcp (v6)	ALLOW IN	Anywhere (v6)	
[8]	80/tcp (v6)	ALLOW IN	Anywhere (v6)	
[9]	23/tcp (v6)	DENY IN	Anywhere (v6)	# Block Telnet

```
ubuntu@ubuntu-server:~/core102_lab4/web
$ sudo ufw status numbered
Status: active

To
From
--
--
[ 1] 8080/tcp
N   Anywhere
[ 2] 80/tcp
N   Anywhere
[ 3] 22/tcp
N   192.168.56.0/24
02  SSH
[ 4] 8080/tcp
N   192.168.56.20
02  test web
[ 5] 23/tcp
N   Anywhere
Telnet
[ 6] 8080/tcp (v6)
N   Anywhere (v6)
[ 7] 80/tcp (v6)
N   Anywhere (v6)
[ 8] 23/tcp (v6)
N   Anywhere (v6)

Action
-----
ALLOW I
ALLOW I
ALLOW I
# CORE1
ALLOW I
# CORE1
DENY IN
# Block
ALLOW I
ALLOW I
DENY IN
# Block
```

Figure 10: Numbered UFW rules before testing.

Reading the list. Rules 3, 4, 5 and 6 are the policy I intended: SSH from the lab subnet, the web test from the two client addresses, and Telnet denied. Rules 1 and 2 (ports 8080 and 80 open to Anywhere) and their IPv6 versions, rules 7 and 8, were broader than the policy I designed. UFW checks rules from the top and the first match wins, so rule 1 effectively opened port 8080 to every host and made the scoped rules 4 and 6 pointless. I treated these as a finding, tested with that in mind in Part 4, and removed them during cleanup in Part 8. The Telnet deny also appears twice, once for IPv4 and once for IPv6, because UFW writes both by default.

5. Part 4: Rule Verification

With the firewall active I ran the four checks from the manual on the Kali client, using the server address 192.168.56.102.

```
nc -vz 192.168.56.102 22
nc -vz 192.168.56.102 8080
nc -vz -w 3 192.168.56.102 23
curl http://192.168.56.102:8080/
```

```
ubuntu@ubuntu-server: ~ -core102_lab4/web

Session Actions Edit View Help

ubuntu@ubuntu-server:~/core102_lab4/web$ echo 'CORE102 authorised firewall test' > ~/core102_lab4/web/index.html
ubuntu@ubuntu-server:~/core102_lab4/web$ python3 -m http.server 8080 --bind 0.0.0.0
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) .
..
192.168.56.101 -- [29/Sep/2026 03:43:29] "GET / HTTP/1.1" 200 -
192.168.56.101 -- [29/Sep/2026 04:26:40] "GET / HTTP/1.1" 200 -

ubuntu@ubuntu-server:~/core102_lab4/web$ sudo ufw status numbered
[sudo: authenticate] Password:
Status: active

To Action Fr
--
[ 1] 8080/tcp ALLOW IN An
ywhere
[ 2] 80/tcp ALLOW IN An
ywhere
[ 3] 22/tcp ALLOW IN 19
2.168.56.0/24 # CORE102 SSH
[ 4] 8080/tcp ALLOW IN 19
2.168.56.20 # CORE102 test web
[ 5] 23/tcp DENY IN An
ywhere # Block Telnet
[ 6] 8080/tcp ALLOW IN 19
2.168.56.101 # CORE102 test web
[ 7] 8080/tcp (v6) ALLOW IN An
ywhere (v6)
[ 8] 80/tcp (v6) ALLOW IN An
ywhere (v6)
[ 9] 23/tcp (v6) DENY IN An
ywhere (v6) # Block Telnet

(kali@kali) [~/Desktop]
$ nc -vz 192.168.56.102 8080
192.168.56.102: inverse host lookup failed:
me lookup failure
(UNKNOWN) [192.168.56.102] 22 (ssh) open

(kali@kali) [~/Desktop]
$ nc -vz 192.168.56.102 8080
^[[3~^[[2~^[[6~^[[B^[[F^[[A192.168.56.102: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.56.102] 8080 (http-alt) open

(kali@kali) [~/Desktop]
$ nc -vz -w 3 192.168.56.102 23
192.168.56.102: inverse host lookup failed:
me lookup failure
(UNKNOWN) [192.168.56.102] 23 (telnet) : Connection timed out

(kali@kali) [~/Desktop]
$ curl http://192.168.56.102:8080/
CORE102 authorised firewall test

(kali@kali) [~/Desktop]
$
```

```

ubuntu@ubuntu-server:~/core102_lab4/web$ sudo journalctl -k --since '10 minutes ago' | grep -i ufw | tail -30
Sep 29 04:23:32 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=64 TOS=0x00 PREC=0x00 TTL=1 ID=14263 DF PROTO=TCP SPT=46320 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:23:33 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=60 TOS=0x00 PREC=0x00 TTL=1 ID=14264 DF PROTO=TCP SPT=46320 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:23:38 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=64 TOS=0x00 PREC=0x00 TTL=1 ID=16126 DF PROTO=TCP SPT=52454 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:23:39 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=60 TOS=0x00 PREC=0x00 TTL=1 ID=16127 DF PROTO=TCP SPT=52454 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:24:00 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=64 TOS=0x00 PREC=0x00 TTL=1 ID=14263 DF PROTO=TCP SPT=46320 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0

(kali@kali)-[~/Desktop]
$ nc -vz 192.168.56.102 22
192.168.56.102: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.56.102] 22 (ssh) open

(kali@kali)-[~/Desktop]
$ nc -vz 192.168.56.102 8080
^[[3~^[[2~^[[6~^[[B^[[F^[[A192.168.56.102: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.56.102] 8080 (http-alt) open

(kali@kali)-[~/Desktop]
$ nc -vz -w 3 192.168.56.102 23
192.168.56.102: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.56.102] 23 (telnet) : Connection timed out

(kali@kali)-[~/Desktop]
$ curl http://192.168.56.102:8080/
CORE102 authorised firewall test

(kali@kali)-[~/Desktop]
$

```

Figure 11: Client-side tests after UFW was enabled (ports 22, 8080 and 23, then curl).

The Telnet test uses `-w 3` because a denied packet is dropped silently. There is no refusal message, so without a timeout netcat would wait for a long time.

On the server I checked the rules, the kernel log and the packet counters to connect each result to the rule that produced it.

```

sudo ufw status numbered
sudo journalctl -k --since '10 minutes ago' | grep -i ufw | tail -30
sudo iptables -L -n -v --line-numbers

```

```

ubuntu@ubuntu-server:~/core102_lab4/web$ sudo journalctl -k --since '10 minutes ago' | grep -i ufw | tail -30
Sep 29 04:23:32 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=64 TOS=0x00 PREC=0x00 TTL=1 ID=14263 DF PROTO=TCP SPT=46320 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:23:33 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=60 TOS=0x00 PREC=0x00 TTL=1 ID=14264 DF PROTO=TCP SPT=46320 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:23:38 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=64 TOS=0x00 PREC=0x00 TTL=1 ID=16126 DF PROTO=TCP SPT=52454 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:23:39 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=60 TOS=0x00 PREC=0x00 TTL=1 ID=16127 DF PROTO=TCP SPT=52454 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0
Sep 29 04:24:00 ubuntu-server kernel: [UFW BLOCK] IN=enp0s8 OUT= MAC=08:00:27:b9:fd:97:08:00:27:0a:6f:b3:08:00 SRC=192.168.56.101 DST=192.168.56.102 LEN=64 TOS=0x00 PREC=0x00 TTL=1 ID=14263 DF PROTO=TCP SPT=46320 DPT=5355 WINDOW=64240 RES=0x00 SYN URGP=0

```

Figure 12: UFW log entries and iptables counters on the server after the tests.

Test	Expected	Observed	Matching rule or explanation
SSH from client	Allowed		Rule 3: allow 22/tcp from 192.168.56.0/24. The client is inside that subnet.
HTTP 8080 from authorised client	Allowed		Rule 6: allow 8080/tcp from 192.168.56.101, the Kali client.
HTTP 8080 from other client	Denied		Only the authorised client addresses are permitted. Any other source falls through to the default deny once the broad rule 1 is removed.
Telnet 23	Denied		Rule 5: explicit deny for 23/tcp. The packet is dropped and logged as a UFW BLOCK entry with destination port 23.

6. Part 5: Safe iptables Chain Exercise

This exercise builds a custom chain without attaching it to INPUT, OUTPUT or FORWARD, so it demonstrates rule structure without changing how live traffic flows.

```
sudo iptables -N CORE102_LAB
sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP
sudo iptables -A CORE102_LAB -p tcp --dport 22 -j ACCEPT
sudo iptables -L CORE102_LAB -n -v --line-numbers
sudo iptables -S CORE102_LAB
sudo iptables -F CORE102_LAB
sudo iptables -X CORE102_LAB
sudo iptables -L CORE102_LAB 2>&1 || echo "Custom chain removed"
```

```
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -N CORE102_LAB
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -A CORE102_LAB -p tcp --dport 22 -j ACCEPT
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -L CORE102_LAB -n -v --line-numbers
Chain CORE102_LAB (0 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DROP      tcp  --  *      *       0.0.0.0/0  0.0.0.0/0
tcp dpt:23
2      0      0 ACCEPT    tcp  --  *      *       0.0.0.0/0  0.0.0.0/0
tcp dpt:22
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -S CORE102_LAB
-N CORE102_LAB
-A CORE102_LAB -p tcp -m tcp --dport 23 -j DROP
-A CORE102_LAB -p tcp -m tcp --dport 22 -j ACCEPT
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -F CORE102_LAB
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -X CORE102_LAB
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -L CORE102_LAB 2>&1 || echo "Custom chain removed"
iptables: No chain/target/match by that name.
Custom chain removed
ubuntu@ubuntu-server:~/core102_lab4/web$ sudo iptables -L CORE102_LAB 2>&1 || echo "Custom chain removed"
iptables: No chain/target/match by that name.
Custom chain removed
ubuntu@ubuntu-server:~/core102_lab4/web$
```

Figure 13: Creating, inspecting and removing the CORE102_LAB chain.

Rule anatomy

`sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP` breaks down as follows:

Part of the command	Element	Meaning
<code>iptables</code>	Tool	The classic Linux packet filtering command
<code>-A CORE102_LAB</code>	Action and chain	Append a rule to the end of the chain named <code>CORE102_LAB</code>
<code>-p tcp</code>	Protocol match	Only match TCP packets
<code>--dport 23</code>	Destination-port match	Only match packets going to port 23 (Telnet)
<code>-j DROP</code>	Target	Silently discard any packet that matched. The second rule uses <code>-j ACCEPT</code> for port 22 instead

The other commands in the exercise do the following. `-N` creates a new chain. `-L` lists its rules with packet counters and line numbers. `-S` prints the same rules in the form of the commands that would recreate them. `-F` flushes (empties) the chain, and `-X` deletes the empty chain.

Why the chain has no effect

Packets only travel through the built-in chains. A custom chain is like a subroutine that nothing calls until one of the built-in chains contains a rule whose target is that chain (for example `-j CORE102_LAB`). I never added such a jump, so no packet ever reached `CORE102_LAB`. Its packet counters stayed at zero in the listing, which supports that.

At the end I flushed and deleted the chain. The final listing reported that the chain does not exist, and the script printed Custom chain removed, so nothing from this exercise was left behind.

7. Part 6: SSH Key Authentication

For key authentication I generated a pair on the Kali client. The private key stays on the client and never leaves it. The public key goes on the server, and it is safe to share because it can only be used to verify a login, not to make one.

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/core102_ed25519 -C "CORE102 authorised lab"
ssh-keygen -lf ~/.ssh/core102_ed25519.pub
ssh-copy-id -i ~/.ssh/core102_ed25519.pub ubuntu@192.168.56.102
ssh -i ~/.ssh/core102_ed25519 ubuntu@192.168.56.102
```

I use `Ed25519` and `-a 64` adds extra key-derivation rounds so that a stolen key file is harder to crack. The key is protected by a passphrase that I chose myself. The restricted permissions on `~/.ssh` matter because SSH ignores keys stored in folders that other users can read.

```

(kali㉿kali)-[~/Desktop]
$ mkdir -p ~/.ssh

(kali㉿kali)-[~/Desktop]
$ chmod 700 ~/.ssh

(kali㉿kali)-[~/Desktop]
$ ssh-keygen -t ed25519 -a 64 -f ~/.ssh/core102_ed25519 -C "CORE102 authorised lab"
Generating public/private ed25519 key pair.
Enter passphrase for "/home/kali/.ssh/core102_ed25519" (empty for no passphrase):
Enter same passphrase again:
Passphrases do not match. Try again.
Enter passphrase for "/home/kali/.ssh/core102_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/core102_ed25519
Your public key has been saved in /home/kali/.ssh/core102_ed25519.pub
The key's fingerprint is:
SHA256:RKMM51Ii8s978EW/E79wqLRoNeIpbWFIGTtqgvG6F88 CORE102 authorised lab
The key's randomart image is:
+--[ED25519 256]--+
|      .o          |
|     .o.o         |
|    .o.o+=.       |
|   .o.=B..        |
|  .o.o.o..S.      |
| o =oo.+ + o.     |
| + +o* *..o+.     |
| . . E.Oo ooo.    |
| .o .=. o ...     |
+---[SHA256]-----+

(kali㉿kali)-[~/Desktop]
$

```

Figure 14: Key generation on the client. The passphrase is not displayed, and the private key is never shown.

```

(kali㉿kali)-[~/Desktop]
(kali㉿kali)-[~/Desktop]
$ ssh-keygen -lf ~/.ssh/core102_ed25519.pub
256 SHA256:RKMM51Ii8s978EW/E79wqLRoNeIpbWFIGTtqgvG6F88 CORE102 authorised lab (ED25519)

(kali㉿kali)-[~/Desktop]
$ ssh-copy-id -i ~/.ssh/core102_ed25519.pub ubuntu@192.168.56.102
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/core102_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any
that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now
it is to install the new keys
Enter passphrase for key '/home/kali/.ssh/id_ed25519':

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /home/kali/.ssh/core102_ed25519 'ubuntu
@192.168.56.102'"
and check to make sure that only the key(s) you wanted were added.

```

Public-key fingerprint

```

(kali㉿kali)-[~/Desktop]
$ ssh-keygen -lf ~/.ssh/core102_ed25519.pub
256 SHA256:RKMM51Ii8s978EW/E79wqLRoNeIpbWFIGTtqgvG6F88 CORE102 authorised lab (ED25519)

```

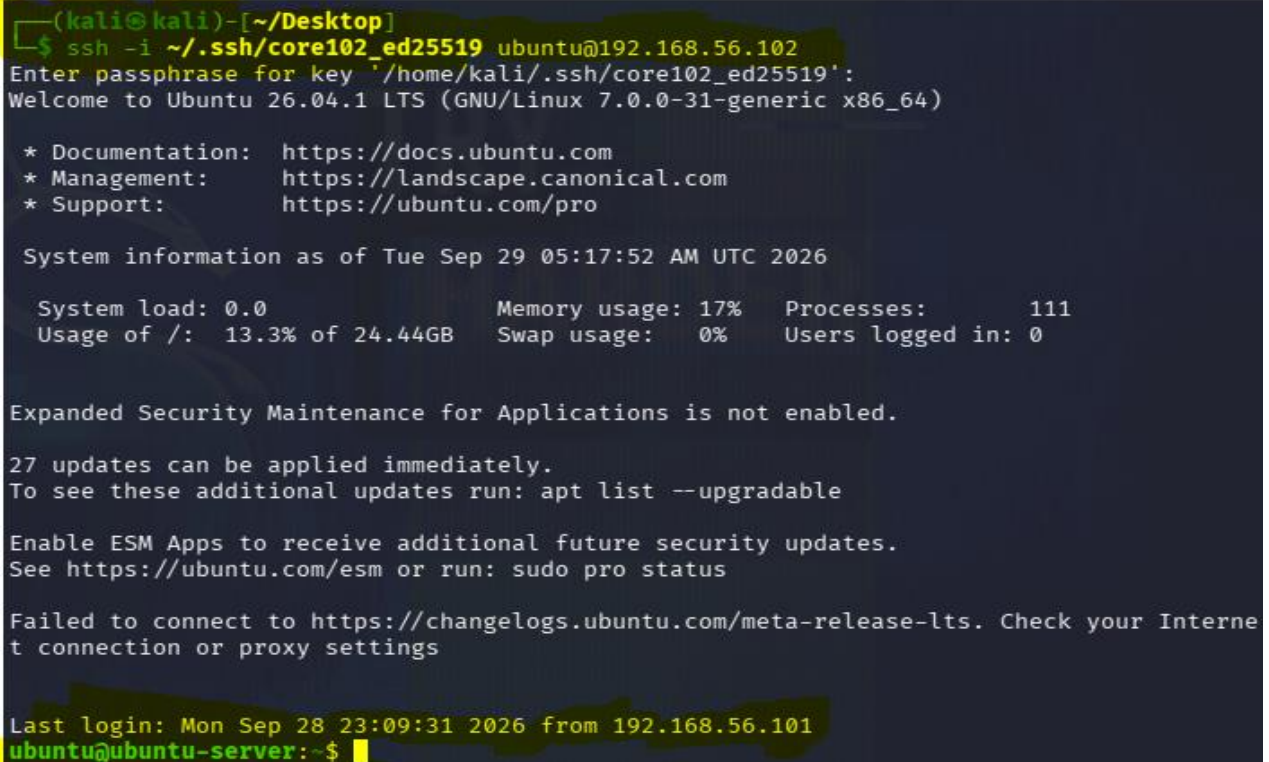
Figure 15: Fingerprint of the public key.

Problem during key deployment

My first attempt to copy the key used the account name from the manual, `student@192.168.56.102`. It failed after three password prompts with `Permission denied (publickey,password)`. The cause was not the password. My server has no account called `student`, because the account on `ubuntu-server` is `ubuntu`, so no password could ever have succeeded. I repeated the command with the correct account:

```
ssh-copy-id -i ~/.ssh/core102_ed25519.pub ubuntu@192.168.56.102
```

This time it asked for the password of the `ubuntu` account on the server, which is needed once because password login was still enabled, and it installed the key into `~/.ssh/authorized_keys` on the server.



```
(kali@kali)-[~/Desktop]
$ ssh -i ~/.ssh/core102_ed25519 ubuntu@192.168.56.102
Enter passphrase for key '/home/kali/.ssh/core102_ed25519':
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-31-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 29 05:17:52 AM UTC 2026

System load: 0.0           Memory usage: 17%        Processes:      111
Usage of /: 13.3% of 24.44GB Swap usage: 0%          Users logged in: 0

Expanded Security Maintenance for Applications is not enabled.

27 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

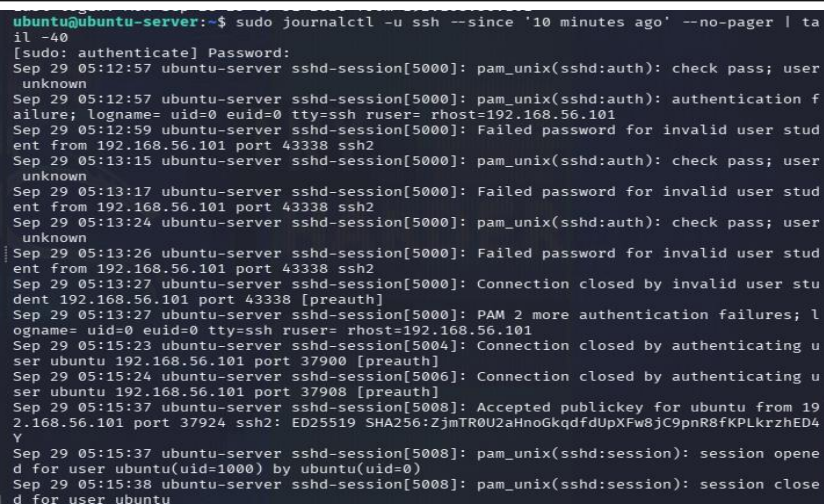
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Mon Sep 28 23:09:31 2026 from 192.168.56.101
ubuntu@ubuntu-server:~$
```

Figure 16: Successful `ssh-copy-id` showing that one key was added.

Login with the key and server-side validation

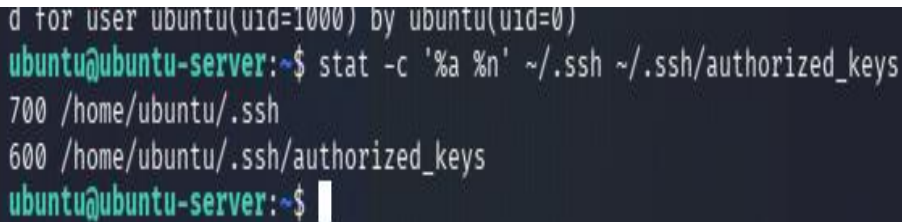
I then logged in with the key. The prompt asked for the passphrase of the key, not the account password, which shows the key was accepted, and the prompt changed to `ubuntu@ubuntu-server`.



```
ubuntu@ubuntu-server:~$ sudo journalctl -u ssh --since '10 minutes ago' --no-pager | tail -40
[sudo: authenticate] Password:
Sep 29 05:12:57 ubuntu-server sshd-session[5000]: pam_unix(sshd:auth): check pass; user unknown
Sep 29 05:12:57 ubuntu-server sshd-session[5000]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.101
Sep 29 05:12:59 ubuntu-server sshd-session[5000]: Failed password for invalid user student from 192.168.56.101 port 43338 ssh2
Sep 29 05:13:15 ubuntu-server sshd-session[5000]: pam_unix(sshd:auth): check pass; user unknown
Sep 29 05:13:17 ubuntu-server sshd-session[5000]: Failed password for invalid user student from 192.168.56.101 port 43338 ssh2
Sep 29 05:13:24 ubuntu-server sshd-session[5000]: pam_unix(sshd:auth): check pass; user unknown
Sep 29 05:13:26 ubuntu-server sshd-session[5000]: Failed password for invalid user student from 192.168.56.101 port 43338 ssh2
Sep 29 05:13:27 ubuntu-server sshd-session[5000]: Connection closed by invalid user student 192.168.56.101 port 43338 [preauth]
Sep 29 05:13:27 ubuntu-server sshd-session[5000]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.101
Sep 29 05:15:23 ubuntu-server sshd-session[5004]: Connection closed by authenticating user ubuntu 192.168.56.101 port 37900 [preauth]
Sep 29 05:15:24 ubuntu-server sshd-session[5006]: Connection closed by authenticating user ubuntu 192.168.56.101 port 37908 [preauth]
Sep 29 05:15:37 ubuntu-server sshd-session[5008]: Accepted publickey for ubuntu from 192.168.56.101 port 37924 ssh2: ED25519 SHA256:ZjmTR0U2aHnoGkqdfDUpXFw8jC9pnR8fKPLkrzhED4Y
Sep 29 05:15:37 ubuntu-server sshd-session[5008]: pam_unix(sshd:session): session opened for user ubuntu(uid=1000) by ubuntu(uid=0)
Sep 29 05:15:38 ubuntu-server sshd-session[5008]: pam_unix(sshd:session): session closed for user ubuntu
```

On the server I checked the authentication log and the permissions on the key files:

```
sudo journalctl -u ssh --since '10 minutes ago' --no-pager | tail -40
stat -c '%a %n' ~/.ssh ~/.ssh/authorized_keys
```



```
d for user ubuntu(uid=1000) by ubuntu(uid=0)
ubuntu@ubuntu-server:~$ stat -c '%a %n' ~/.ssh ~/.ssh/authorized_keys
700 /home/ubuntu/.ssh
600 /home/ubuntu/.ssh/authorized_keys
ubuntu@ubuntu-server:~$
```

Figure 18: Server log showing Accepted publickey for ubuntu from 192.168.56.101, and permissions 700 and 600.

The log line Accepted publickey for ubuntu from 192.168.56.101 confirms that the server authenticated the session with the key. The permissions of 700 on the .ssh folder and 600 on authorized_keys are what SSH expects.

8. Part 7: SSH Server Hardening

I only started this part after the key login in Part 6 had worked, because turning off password login before that point could have locked me out. I kept my original SSH session open for the whole of this part and did all testing in a second one.

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.core102.bak
sudo nano /etc/ssh/sshd_config.d/90-core102-hardening.conf
```

The drop-in file contains:

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

Setting	Effect
PermitRootLogin no	Nobody can log in over SSH as root
PubkeyAuthentication yes	Key-based login stays enabled
PasswordAuthentication no	Password login is switched off, so guessing passwords no longer works
KbdInteractiveAuthentication no	Closes the keyboard-interactive route, which can otherwise behave like a password prompt

Figure 19: Backup command and contents of the hardening drop-in file.

I validated the configuration before applying it. `sudo sshd -t` prints nothing when the file is valid, and I would not have reloaded if it had reported an error. I then reloaded the service, which applies the change without dropping the session I already had open.

```

ubuntu@ubuntu-server: ~
Session Actions Edit View Help

ubuntu@ubuntu-server:~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.core102.bak
ubuntu@ubuntu-server:~$ sudo nano /etc/ssh/sshd_config.d/90-core102-hardening.conf
ubuntu@ubuntu-server:~$ sudo sshd -t
ubuntu@ubuntu-server:~$ sudo systemctl reload ssh
ubuntu@ubuntu-server:~$ sudo systemctl --no-pager status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Mon 2026-09-28 17:02:26 UTC; 12h ago
 Invocation: 148ddce6a6b94d58b4393b4e40d44c88
 TriggeredBy: ● ssh.socket
             Docs: man:sshd(8)
                  man:sshd_config(5)
   Process: 1347 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 5287 ExecReload=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 5290 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS)
 Main PID: 1350 (sshd)
   Tasks: 1 (limit: 1885)
  Memory: 5.5M (peak: 9M)
    CPU: 1.593s
   CGroup: /system.slice/ssh.service
           └─1350 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 29 05:15:37 ubuntu-server sshd-session[5008]: Accepted publickey for ubuntu from 192.168.56.101 port 37924 ssh2: ED25519 SHA256:ZjmTR0U2aHnoGkqdfUpXF...fKPLkrzhED4Y
Sep 29 05:15:37 ubuntu-server sshd-session[5008]: pam_unix(sshd:session): session opened for user ubuntu(uid=1000) by ubuntu(uid=0)
Sep 29 05:15:38 ubuntu-server sshd-session[5008]: pam_unix(sshd:session): session closed for user ubuntu
Sep 29 05:17:52 ubuntu-server sshd-session[5101]: Accepted publickey for ubuntu from 192.168.56.101 port 51734 ssh2: ED25519 SHA256:RKMm51Ii8s978EW/E79wqL...IGTtqgvG6F88
Sep 29 05:17:52 ubuntu-server sshd-session[5101]: pam_unix(sshd:session): session opened for user ubuntu(uid=1000) by ubuntu(uid=0)
Sep 29 05:30:42 ubuntu-server systemd[1]: Reloading ssh.service - OpenBSD Secure Shell server ...
Sep 29 05:30:42 ubuntu-server sshd[1350]: Received SIGHUP; restarting.
Sep 29 05:30:42 ubuntu-server systemd[1]: Reloaded ssh.service - OpenBSD Secure Shell server.
Sep 29 05:30:42 ubuntu-server sshd[1350]: Server listening on 0.0.0.0 port 22.
Sep 29 05:30:42 ubuntu-server sshd[1350]: Server listening on :: port 22.
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ubuntu-server:~$

```

```

sudo sshd -t
sudo systemctl reload ssh
sudo systemctl --no-pager status ssh

```

```

ubuntu@ubuntu-server: ~
Session Actions Edit View Help

ubuntu@ubuntu-server:~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.core102.
ubuntu@ubuntu-server:~$ sudo nano /etc/ssh/sshd_config.d/90-core102-hardening.conf
ubuntu@ubuntu-server:~$ sudo sshd -t
ubuntu@ubuntu-server:~$ sudo systemctl reload ssh
ubuntu@ubuntu-server:~$ sudo systemctl --no-pager status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enable
   Active: active (running) since Mon 2026-09-28 17:02:26 UTC; 12h ago
 Invocation: 148ddce6a6b94d58b4393b4e40d44c88
 TriggeredBy: ● ssh.socket
             Docs: man:sshd(8)
                  man:sshd_config(5)
   Process: 1347 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 5287 ExecReload=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 5290 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS)
 Main PID: 1350 (sshd)
   Tasks: 1 (limit: 1885)
  Memory: 5.5M (peak: 9M)
    CPU: 1.593s
   CGroup: /system.slice/ssh.service
           └─1350 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 29 05:15:37 ubuntu-server sshd-session[5008]: Accepted publickey for ubuntu fr
Sep 29 05:15:37 ubuntu-server sshd-session[5008]: pam_unix(sshd:session): session
Sep 29 05:15:38 ubuntu-server sshd-session[5008]: pam_unix(sshd:session): session
Sep 29 05:17:52 ubuntu-server sshd-session[5101]: Accepted publickey for ubuntu fr
Sep 29 05:17:52 ubuntu-server sshd-session[5101]: pam_unix(sshd:session): session
Sep 29 05:30:42 ubuntu-server systemd[1]: Reloading ssh.service - OpenBSD Secure S
Sep 29 05:30:42 ubuntu-server sshd[1350]: Received SIGHUP; restarting.
Sep 29 05:30:42 ubuntu-server systemd[1]: Reloaded ssh.service - OpenBSD Secure Sh
Sep 29 05:30:42 ubuntu-server sshd[1350]: Server listening on 0.0.0.0 port 22.
Sep 29 05:30:42 ubuntu-server sshd[1350]: Server listening on :: port 22.
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ubuntu-server:~$

```

```

(kali@kali)~[~/Desktop]
$ ssh -i ~/.ssh/core102_ed25519 ubuntu@192.168.56.102
Enter passphrase for key '/home/kali/.ssh/core102_ed25519':
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-31-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 29 05:38:20 AM UTC 2026

System load:  0.0          Processes:           109
Usage of /:   13.3% of 24.44GB  Users logged in:    0
Memory usage: 16%          IPv4 address for enp0s8: 192.168.56.
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

27 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Ch
ternet connection or proxy settings

Last login: Tue Sep 29 05:17:53 2026 from 192.168.56.101
ubuntu@ubuntu-server:~$

```

Figure 20: Configuration test, reload and service status.

Ubuntu reads the files in the drop-in folder in alphabetical order and the first value it finds for a setting wins, so a file with an earlier name could silently override mine. To be sure my settings were the ones in force, I checked the effective configuration:

```
sudo sshd -T | grep -Ei 'passwordauthentication|permitrootlogin|pubkeyauthentication'
```

Finally, from a second terminal on the client, I opened a new session with the key while the first one stayed connected:

```
ssh -i ~/.ssh/core102_ed25519 ubuntu@192.168.56.102
```

The key login still worked after the change, so access was not lost.

```
sudo cp /etc/ssh/sshd_config.core102.bak /etc/ssh/sshd_config
sudo rm /etc/ssh/sshd_config.d/90-core102-hardening.conf
sudo sshd -t
sudo systemctl reload ssh
```

This restores the original configuration from the backup, removes my drop-in, checks that the result is valid, and reloads the service.

9. Part 8: Final Exposure Review and Cleanup

I stopped the temporary Python web server with Ctrl+C in its terminal. Then I listed the UFW rules with their numbers and deleted the temporary 8080 rules. Two details mattered here. The rule numbers shift after every deletion, so I deleted from the highest number to the lowest and re-listed the rules after each one. Also, the list contained several 8080 entries, not just one: the IPv6 twin of the broad rule (7), my Kali client rule (6), the manual's example client rule (4), and the broad IPv4 rule (1).

```
sudo ufw status numbered
sudo ufw delete 7
sudo ufw status numbered
sudo ufw delete 6
sudo ufw status numbered
sudo ufw delete 4
sudo ufw status numbered
sudo ufw delete 1
sudo ufw status numbered
```

```
tu@ubuntu-server: ~
ubuntu@ubuntu-server:~$ sudo ufw status numbered
[sudo: authenticate] Password:
Status: active

      To
      --
[ 1] 8080/tcp
[ 2] 80/tcp
[ 3] 22/tcp
SH
[ 4] 8080/tcp
est web
[ 5] 23/tcp
net
[ 6] 8080/tcp
est web
[ 7] 8080/tcp (v6)
[ 8] 80/tcp (v6)
[ 9] 23/tcp (v6)
net

      Action      From
      --
[ 1] ALLOW IN     Anywhere
[ 2] ALLOW IN     Anywhere
[ 3] ALLOW IN     192.168.56.0/24      # CORE102 S
[ 4] ALLOW IN     192.168.56.20       # CORE102 t
[ 5] DENY IN     Anywhere            # Block Tel
[ 6] ALLOW IN     192.168.56.101      # CORE102 t
[ 7] ALLOW IN     Anywhere (v6)
[ 8] ALLOW IN     Anywhere (v6)
[ 9] DENY IN     Anywhere (v6)      # Block Tel

ubuntu@ubuntu-server:~$ sudo ufw delete 1
Deleting:
  allow 8080/tcp
Proceed with operation (y/n)? y
Rule deleted
ubuntu@ubuntu-server:~$ sudo ufw status numbered
Status: active

      To
      --
[ 1] 80/tcp
[ 2] 22/tcp
SH

      Action      From
      --
[ 1] ALLOW IN     Anywhere
[ 2] ALLOW IN     192.168.56.0/24      # CORE102 S
```

Figure 23: UFW rules listed after each deletion, ending with no 8080 rules.

The two rules for port 80 (rules 2 and 8 in the earlier list) were not part of this lab. Nothing is listening on port 80, as the socket inventory shows, so they expose nothing today.

I then proved that port 8080 is closed, re-checked the full socket list and confirmed that SSH still worked from the client.

```
sudo ss -tulpn
sudo ss -ltnp | grep ':8080' || echo "Temporary web service stopped"
```

```
--
[ 1] 80/tcp      ALLOW IN    Anywhere    # CORE102 S
[ 2] 22/tcp      ALLOW IN    192.168.56.0/24
[ 3] 8080/tcp     ALLOW IN    192.168.56.20    # CORE102 t
[ 4] 23/tcp      DENY IN     Anywhere    # Block Tel
[ 5] 8080/tcp     ALLOW IN    192.168.56.101   # CORE102 t
[ 6] 8080/tcp (v6) ALLOW IN    Anywhere (v6)
[ 7] 80/tcp (v6)  ALLOW IN    Anywhere (v6)
[ 8] 23/tcp (v6)  DENY IN     Anywhere (v6)    # Block Tel
net

ubuntu@ubuntu-server:~$ sudo ufw delete 3
Deleting:
allow from 192.168.56.20 to any port 8080 proto tcp comment 'CORE102 test web'
Proceed with operation (y/n)? y
Rule deleted
ubuntu@ubuntu-server:~$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 80/tcp      ALLOW IN    Anywhere
[ 2] 22/tcp      ALLOW IN    192.168.56.0/24    # CORE102 SSH
[ 3] 23/tcp      DENY IN     Anywhere           # Block Telnet
[ 4] 8080/tcp     ALLOW IN    192.168.56.101     # CORE102 test web
[ 5] 8080/tcp (v6) ALLOW IN    Anywhere (v6)
[ 6] 80/tcp (v6)  ALLOW IN    Anywhere (v6)
[ 7] 23/tcp (v6)  DENY IN     Anywhere (v6)      # Block Telnet
ubuntu@ubuntu-server:~$
```

Figure 24: Final socket inventory, and confirmation that nothing listens on port 8080.

```

Rule deleted
ubuntu@ubuntu-server:~$ sudo ufw status numbered
Status: active

    To      Action      From
    --      -
[ 1] 80/tcp    ALLOW IN    Anywhere
[ 2] 22/tcp    ALLOW IN    192.168.56.0/24      # CORE102 SSH
[ 3] 23/tcp    DENY IN     Anywhere             # Block Telnet
[ 4] 8080/tcp   ALLOW IN    192.168.56.101       # CORE102 test web
[ 5] 8080/tcp (v6) ALLOW IN    Anywhere (v6)
[ 6] 80/tcp (v6) ALLOW IN    Anywhere (v6)
[ 7] 23/tcp (v6) DENY IN     Anywhere (v6)       # Block Telnet

ubuntu@ubuntu-server:~$ sudo ufw delete 4
Deleting:
allow from 192.168.56.101 to any port 8080 proto tcp comment 'CORE102 test web'
Proceed with operation (y/n)? y
Rule deleted
ubuntu@ubuntu-server:~$ sudo ufw status numbered
Status: active

    To      Action      From
    --      -
[ 1] 80/tcp    ALLOW IN    Anywhere
[ 2] 22/tcp    ALLOW IN    192.168.56.0/24      # CORE102 SSH
[ 3] 23/tcp    DENY IN     Anywhere             # Block Telnet
[ 4] 8080/tcp (v6) ALLOW IN    Anywhere (v6)
[ 5] 80/tcp (v6) ALLOW IN    Anywhere (v6)
[ 6] 23/tcp (v6) DENY IN     Anywhere (v6)       # Block Telnet

ubuntu@ubuntu-server:~$

```

Figure 25: SSH login from the client after cleanup.

Comparison with the baseline

Service	Baseline	Final	Explanation
Port 8080 web test	Not present	Removed	Temporary test service. Stopped, and its firewall rules deleted
SSH (22/tcp)	Open to all, password login	Open to the lab subnet only, key login only	Needed for administration. Now restricted by UFW and hardened by the drop-in file
DHCP (67/udp)	Listening on all interfaces	Unchanged	Left as found. It is only justified if this server issues leases for the lab network
DNS (53), NTP control (323)	Loopback only	Unchanged	Not reachable from the network, so they add no network exposure
Firewall	Inactive, all policies ACCEPT	Active, default deny incoming	Only explicitly approved traffic is accepted

Residual risk

- The DHCP server still listens on all interfaces. UFW now blocks incoming traffic that has no allow rule, but DHCP clients need port 67, so I did not add a rule for it and left the decision to the instructor.
- SSH is reachable by any host on 192.168.56.0/24, not just the client. Narrowing the rule to the client address would be a further improvement.
- The security of the server now depends on the private key and its passphrase. If the key were copied and the passphrase were weak, the protection would be lost.

As the manual instructs, I have not deleted the client private key. I will keep it until the instructor confirms that evidence review is complete.

10. Reflection Questions

1. Why must SSH access be allowed before a default-deny firewall is enabled?

A default-deny policy blocks every incoming connection that has no rule allowing it. If I had enabled it first, the rule that lets my own SSH traffic in would not yet exist, and new connections to the server would be refused. On a remote machine that means being locked out with only the console to recover. Adding the allow rule first means the firewall never blocks the path I am using.

2. What is the difference between an open port and an authorised service?

An open port only means that something is listening and accepting connections. An authorised service is one that is documented, needed and permitted by the security policy. In my baseline, both SSH and DHCP had open ports, but only SSH had a clear administrative purpose. A port can be open because of a forgotten or unwanted program, and that is the situation a service inventory is meant to catch.

3. Why does the server need the public key while the client must protect the private key?

The two keys work as a pair. The server uses the public key to check a signature, and it cannot be used to create one, so sharing it does no harm. The private key creates that signature and so proves who is logging in. Anyone who holds it, unless it is protected by a passphrase, could log in as me, which is why it stays on the client and never goes into the report or the ZIP file.

4. Which rollback step would recover access after an invalid SSH configuration?

Open the VM console and restore the backup with `sudo cp /etc/ssh/sshd_config.core102.bak /etc/ssh/sshd_config`. Then remove the drop-in file `90-core102-hardening.conf`, run `sudo sshd -t` to confirm the configuration is valid, and reload SSH. The console works even when SSH is down.